Week 5 Project Guide

Tree-Based Models & Ensembles

Role Requirements · Notebook Walkthrough · Submission Guide

Statistical Machine Learning · Fusemachines AI Fellowship · Susan Ghimire

How to use this document: Read the **Your Role** section first — it sets the business context for every decision you will make in the notebook. Then work through the **Notebook Walkthrough** section by section. Before you submit, run through the **Completion Checklist** and the **Submission Requirements** to make sure nothing is missing. All links in this document are clickable.

1 YOUR ROLE

The business scenario you are operating in for this entire assignment

You are a **Data Scientist at a telecom company**. The business is losing customers to churn and needs a production-ready predictive system — not a demo. The analytics team previously tried basic linear models but they failed to capture the complex, non-linear relationships in customer behaviour. You have one week to upgrade the system using tree-based ensembles.

Responsibility	What it means in practice
Build robust models	Ensemble methods (Random Forest, XGBoost) that outperform the naïve baseline on AUROC, Precision, Recall and F1 — not just accuracy.
Prevent data leakage	All preprocessing (scaling, SMOTE, encoding) must happen *inside* a Scikit-Learn Pipeline so it is applied only to training folds during CV.
Tune systematically	Apply Grid Search or Bayesian optimisation to at least one boosting model. Document which hyperparameter had the largest impact.
Explain decisions to stakeholders	Generate SHAP global and local explanations. Write a 2-sentence retention recommendation for a specific at-risk customer.
Ship to production	Save the full fitted pipeline (not just the model) using joblib. Complete a model card before handing off to engineering.

Key challenge: These models are prone to (1) severe overfitting when unconstrained, (2) data leakage if pipelines are not structured correctly, and (3) producing predictions stakeholders do not trust because the model is a "black box." You must build, tune, rigorously evaluate, and **explain** your models before saving them for production.

2 DATASET

Telco Customer Churn — the single source of truth for all experiments

Property	Detail
Source	Kaggle — Telco Customer Churn
Size	7,043 rows · 21 columns · single CSV file
Primary target	**Churn** — Yes / No (binary classification, ~27% positive rate)
Secondary target	**tenure** — months a customer has been with the company (regression)
Key features	Demographics (gender, SeniorCitizen, Partner, Dependents); Services (PhoneService, InternetService, StreamingTV, …); Account (Contract, PaymentMethod, MonthlyCharges, TotalCharges)

Known data quality issues you must handle:

- **TotalCharges dtype bug:** Rows where a customer has just joined have an empty string in TotalCharges rather than a numeric value. Pandas reads the whole column as object. You must coerce it to numeric with errors='coerce', then impute or drop the resulting NaNs.
- **Class imbalance:** Only ~27% of customers churned. A naïve classifier that always predicts "no churn" gets ~73% accuracy — this is the Accuracy Trap you must expose in Q6.
- **Mixed dtypes:** The dataset contains both numeric columns (MonthlyCharges, tenure, TotalCharges) and categorical columns (Contract, PaymentMethod, InternetService, …). These require different preprocessing steps inside your ColumnTransformer.

3 HOW TO USE THE NOTEBOOK

Step-by-step guide for opening, running, and completing the assignment

- Open the notebook shared with you in Google Classroom and load it in Google Colab.
- Run all cells from top to bottom, following the instructions written inside the notebook.
- When done, submit the completed notebook (.ipynb) and the generated pipeline file (telco_churn_pipeline_v1.joblib) back through Google Classroom.

4 SUBMISSION REQUIREMENTS

Exactly what to submit, how, and what will be checked

4.1 Files to Submit

File	Description	Required?
`W5_Assignment.ipynb`	Your completed assignment notebook. Must be fully executed — all cells run top-to-bottom with all outputs and plots visible. All SELF-CHECK cells pass. All ✍■ Reflect cells filled in.	■ **Required**
`telco_churn_pipeline_v1.joblib`	The serialised production pipeline generated in Q17. Must load without errors using joblib.load() and produce correct predictions.	■ **Required**
`model_metadata_v1.json (optional)`	JSON file with model metadata: version, training date, key metrics, feature list. Demonstrates production hygiene.	Optional

4.2 Notebook Quality Standards

- All cells are executed in order from top to bottom — no skipped cells, no cells with errors in the output.
- Every # YOUR CODE HERE has been replaced with working code.
- Every ✍■ **Reflect** markdown cell contains your own reasoning — not a copy-paste from the session notebook or an AI-generated answer.
- All **SELF-CHECK** assert cells pass without errors.
- SHAP plots (Q15 summary plot, Q16 waterfall/force plot) are visible in the output — not empty axes.
- The Model Card (Q18) contains real metric values from your experiments — no placeholder text remains.
- Code is clean and readable — no commented-out dead code blocks left behind.

Do not submit: The raw CSV data file (too large), the session solution notebook, any .pyc files or __pycache__ directories, or files generated by Jupyter checkpoints (.ipynb_checkpoints/). Submit only the two required files listed above.

5 COMMON MISTAKES TO AVOID

Patterns that cause lost marks or incorrect results

■ Using sklearn.pipeline.Pipeline with SMOTE (Q13)

Standard sklearn Pipeline does not know how to restrict SMOTE to training folds only. Always use imblearn.pipeline.Pipeline (imported as ImbPipeline) when SMOTE is a step. If you use the wrong one, your AUROC will be inflated by ~5–15 points and the SELF-CHECK will not catch it.

■ Fitting the scaler on the full dataset before train/test split (Q12–Q14)

If you call ColumnTransformer.fit() on X_train + X_test together, the scaler learns the test set's mean and variance — this is preprocessing leakage. Always split first, then fit the transformer inside the pipeline on X_train only.

■ Not re-running setup after a Colab session timeout or reconnect

Colab runtimes are temporary. After a timeout, disconnect, or manual reconnect, all installed packages and variables are gone. Always re-run the install cell (!pip install …) and then Cell 1 (imports) before continuing. If you see ModuleNotFoundError or NameError, this is almost certainly the cause.

■ Saving only the model object instead of the full pipeline (Q17)

If you save only the XGBClassifier and not the pipeline, the saved model will not include the ColumnTransformer or the SMOTE step. Loading it on new data will fail because the data has not been preprocessed. Save the entire pipeline object.

■ Confusing neg_root_mean_squared_error sign (Q21)

sklearn returns negative values from neg_root_mean_squared_error so that higher = better for all scoring functions. Multiply by −1 before plotting, or the y-axis will be upside-down and the learning curve will appear to improve as more data makes things worse.

■ Writing generic Reflect answers

Answers like 'the model performed well' or 'SHAP explains the predictions' earn no marks. Reflect answers must be specific: name features, quote numbers, reference the business problem. E.g. 'Tenure = 1 month pushed the churn probability up by +0.31 because early-stage customers have not yet built switching costs.'

Remember: The combination of correct code + written reasoning is what separates a model that ships from one that sits in a notebook. A perfect AUROC with empty Reflect cells will not pass. A thoughtful reflection with a minor code bug is much better.
